

Portfolio Project: System Architecture Plan

Case Study: Najm Application

1. Position Your System in the Distributed Landscape

Selected Architecture Model: Three-Tier Client-Server Architecture supported by a Distributed Cloud Infrastructure.

Justification based on System Requirements:

The Najm application possesses a critical mission profile characterized by intermittent yet extremely high-stakes usage. To manage potential Server Overload effectively—especially during regional crises or peak traffic hours caused by adverse weather conditions (such as heavy rainstorms which naturally trigger a massive surge in traffic accidents)—the platform rejects a centralized bottleneck design. A pure Thin-client approach would burden back-end components with primitive presentation rendering under pressure, whereas a pure Fat-client model introduces critical inconsistencies across varying smartphone hardware specs. The chosen Multi-Tier model intelligently leverages modern client-side smartphone capabilities for crisp localized rendering, UI state control, and basic input optimization, while offloading heavy multi-tenant business operations and data state synchronizations to a high-powered, decentralized fleet of network nodes.

2. Define the Multi-tier Architecture & Middleware Bridge

The Najm system organization isolates core computational tasks into three clearly separated structural tiers to enforce absolute task isolation:

- **Presentation Tier (Client/UI):** The native mobile client applications operating on drivers' and field investigators' smartphones. It is responsible entirely for data capture (accident images, geolocations, scanning IDs), local cryptographic formatting, and displaying real-time dispatcher dispatch statuses.
- **Application Tier (Server/Logic):** The engine of the distributed landscape running as isolated microservices on cloud compute nodes. This tier evaluates accident reports, runs dispatch routing algorithms, matches telemetry with geospatial systems, and handles external insurance web hooks.
- **Data Tier (Database/State):** A highly resilient distributed database infrastructure deployed in a Master-Slave replication topology. It logs permanent structural records, legal compliance files, and insurance case histories securely.

The Role of Middleware: Middleware operates as the central neurological bridge that safely binds these distributed segments. By incorporating an advanced API Gateway alongside dedicated asynchronous Message Queues (such as RabbitMQ/Apache Kafka), independent

platform services can exchange massive payloads safely. This middleware ensures low latency by preventing heavy data transfers—such as uploading high-resolution multi-angle accident photos—from clogging or blocking light, time-critical telemetry pipelines (such as reporting urgent GPS coordinates or dispatch notifications).

3. Validation, Exception Handling & Localized Control

Preventing Total UI Collapse (Exception Handling): To ensure that a major infrastructural fault in one tier (e.g., an abrupt database outage or a regional network breakdown) does not propagate into a catastrophic silent failure or a complete freeze of the driver's UI, the system rigidly implements the core software dependability commandment: *'Handle All Exceptions; catch and log everything.'* The Application Tier acts as an exception barrier. If the Data Tier becomes unresponsive, the middleware intercepts the database exception, safely isolates it, records a localized log entry, and passes a graceful, standardized response to the Presentation Tier. The client app catches this message and shifts smoothly into an offline-first caching mechanism, avoiding a crash and maintaining responsiveness.

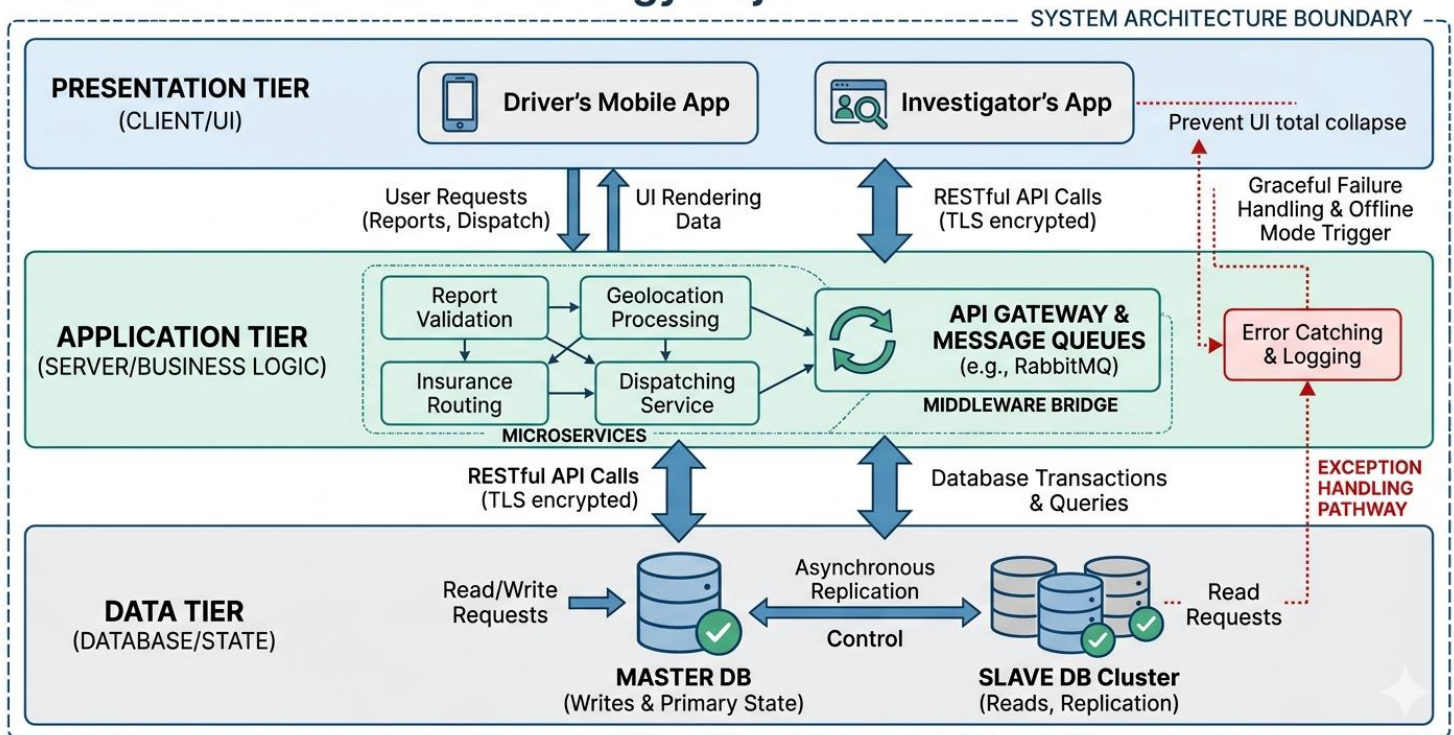
Master-Slave Replication Control: Within the Data Tier, real-time responsiveness and state continuity are achieved using a distinct Master-Slave replication model. The Master Database Node maintains absolute centralized authority over all modification tasks (Write Operations) to protect global data consistency. Concurrently, multiple read-only Slave Nodes execute local queries (Read Operations), such as validating a driver's active policy data or matching user logs. This structure ensures that heavy back-end database analytics never degrade the responsiveness of the front-line accident reporting pipeline.

4. Industry Alignment & Dynamic Scaling

Microservices Synergy: This multi-tier strategy aligns completely with the modern industrial standard of Microservices Architecture. By breaking down the Application Tier into isolated, independently deployable software components (e.g., Geolocation Services, Image Processing, and Dispatching Engine), the engineering team can execute seamless service updates without risk of platform downtime. Crucially, the architecture utilizes cloud-native Auto-scaling. When traffic spikes due to emergency road scenarios, the system automatically scales out specific application nodes on-demand, optimizing infrastructure expenditure and bolstering structural resilience.

5. Multi-tier & Middleware Strategy Diagram

Multi-Tier & Midleware Strategy Najm



Architecture Continuity Analysis:

As clearly demonstrated in the diagram above, isolating the platform into distinct tiers bound by robust middleware completely eliminates any Single Point of Failure (SPOF). This multi-tier configuration directly supports real-world resilience, guaranteeing that front-end driver interfaces remain fully operational and context-aware even during backend disruptions, supporting long-term maintenance and rapid adaptation to modern insurance and emergency compliance markets.